

Pemrograman Berorientasi Objek ala Go: Menguasai Struct, Method, dan Optimasi Pointer

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Golang didesain sebagai bahasa pemrograman yang modular dan efisien. Go tidak memiliki kata kunci `class` maupun mekanisme pewarisan (*inheritance*) seperti bahasa pemrograman berorientasi objek tradisional (Java, C++, atau C#). Sebagai gantinya, Go menggunakan kombinasi **Struct** untuk mendefinisikan model data, **Method** untuk memberikan perilaku (*behavior*), serta **Pointer** untuk mengoptimalkan penggunaan memori dan memanipulasi data secara langsung di alamat memori asli.

Modul ini akan membedah tuntas bagaimana menyusun arsitektur kode berorientasi objek yang bersih, menghindari penyalinan memori yang tidak perlu, dan mencegah *nil pointer dereference* pada aplikasi tingkat produksi.

Sesi 1: OOP Tanpa Class: Menguasai Struct (45 Menit)

`struct` adalah kumpulan dari satu atau lebih field/properti dengan tipe data tertentu yang dibungkus dalam satu nama tipe data baru. Struct bertindak sebagai cetak biru (*blueprint*) data kustom Anda.

1. Deklarasi dan Inisialisasi Struct

Ada beberapa cara menginisialisasi struct di Go:

```

package main

import "fmt"

type Customer struct {
    ID      string
    Name    string
    Email   string
    Age     int
}

func main() {
    // Cara 1: Menggunakan key-value (Sangat direkomendasikan karena aman dari perubahan)
    c1 := Customer{
        ID:      "C-001",
        Name:    "Dika",
        Email:   "dika@danamas.id",
        Age:     28,
    }

    // Cara 2: Positional (Tidak direkomendasikan untuk struct besar)
    c2 := Customer{"C-002", "Budi", "budi@gmail.com", 30}

    fmt.Println(c1.Name) // Mengakses field menggunakan tanda titik (.)
    fmt.Println(c2.Email)
}

```

2. Komposisi Struct (Embedded Struct / Composition)

Go menganut prinsip "*Composition over Inheritance*". Daripada mewarisi sifat dari class induk, kita menanamkan (*embed*) satu struct ke struct lainnya untuk menggunakan kembali kodenya.

```

package main

import "fmt"

type Address struct {
    Street  string
    City    string
    ZipCode string
}

type Vendor struct {
    ID        string
    Name      string
    Address   // Anonymous Field (Struct Embedding)
}

func main() {
    v := Vendor{
        ID:      "VND-99",
        Name:    "Gudang Utama",
        Address: Address{
            Street:  "Jl. Sudirman No. 12",
            City:    "Jakarta Selatan",
            ZipCode: "12190",
        },
    }

    // Promosi Field: Kita bisa mengakses properti Address langsung dari objek Vendor
    fmt.Println("Kota Vendor:", v.City) // Setara dengan: v.Address.City
}

```

3. Struct Tag (Sangat Penting untuk JSON Backend)

Struct tag digunakan untuk memberikan metadata tambahan pada field struct. Metadata ini paling sering dibaca oleh library serialisasi data seperti encoding/json.

```

type Product struct {
    ID      string `json:"id"`
    Name    string `json:"product_name"`
    Price   float64 `json:"price"`
    Stock   int    `json:"qty"`
}

```

Sesi 2: Pointer Deep Dive: Manajemen Alamat Memori (45 Menit)

Secara default, Go adalah bahasa pemrograman yang bertipe **Pass-by-Value**. Artinya, ketika Anda mengirimkan variabel sebagai argumen ke fungsi atau method, Go akan membuat salinan (*copy*) data tersebut di memori (Stack). Untuk data besar, ini tidak efisien. Di sinilah **Pointer** digunakan.

1. Memahami Alamat Memori (&) dan Dereferensi (*)

- **Operator & (Address-of):** Digunakan untuk mendapatkan alamat memori suatu variabel.
- **Operator * (Dereferencing):**
 - Sebagai tipe data (misal `*int` atau `*Customer`): Menandakan variabel tersebut menyimpan alamat memori tipe terkait.
 - Sebagai operator (misal `*pointer`): Mengakses atau mengubah nilai yang ada di dalam alamat memori tersebut.

```
package main

import "fmt"

func main() {
    age := 25
    var ptr *int = &age // ptr menyimpan alamat memori age

    fmt.Println("Nilai age:", age)           // Output: 25
    fmt.Println("Alamat memori age:", ptr) // Output: 0xc0000aa008 (alamat acak)

    *ptr = 30 // Mengubah nilai di alamat memori ptr
    fmt.Println("Nilai age setelah diubah lewat pointer:", age) // Output: 30
}
```

2. Pointer ke Struct

Saat kita memiliki pointer ke sebuah struct, Go memberikan kemudahan akses (*syntactic sugar*). Kita tidak perlu menulis `(*ptrCustomer).Name`, cukup `ptrCustomer.Name`. Go secara otomatis melakukan dereferensi di balik layar.

```

type Employee struct {
    Name string
}

emp := Employee{Name: "Rian"}
ptrEmp := &emp

ptrEmp.Name = "Rian Wijaya" // Otomatis direferensikan oleh Go

```

3. Bahaya Pointer: Nil Pointer Dereference

Sebuah pointer yang tidak diinisialisasi akan menunjuk ke `nil` (Zero Value). Jika Anda mencoba mengakses field dari pointer `nil`, aplikasi akan langsung mengalami crash dengan status **panic**.

```

package main

import "fmt"

type Profile struct {
    Bio string
}

func main() {
    var user *Profile // Nil Pointer

    // Salah dan akan mengakibatkan Panic!
    // fmt.Println(user.Bio)

    // Cara Aman:
    if user != nil {
        fmt.Println(user.Bio)
    } else {
        fmt.Println("Profil belum diinisialisasi")
    }
}

```

Sesi 3: Kolaborasi Struct, Method, dan Pointer (45 Menit)

Kekuatan utama arsitektur pemrograman Go muncul ketika Struct, Method, dan Pointer digabungkan bersama.

1. Pointer Receiver vs Value Receiver dalam Struct

Kapan kita harus menggunakan Pointer Receiver (`func (s *Struct) Method()`)?

1. Jika method perlu mengubah field di dalam struct secara permanen.
2. Jika struct berukuran besar. Menggunakan pointer menghindari overhead menyalin seluruh struct ke stack memori baru setiap kali method dipanggil.

```
package main

import "fmt"

type Cart struct {
    TotalQty    int
    TotalPrice  float64
}

// Value Receiver: Data dicopy, perubahan di dalam method TIDAK mengubah objek asli
func (c Cart) UpdatePriceWrong(newPrice float64) {
    c.TotalPrice = newPrice
}

// Pointer Receiver: Memanipulasi alamat asli, perubahan bersifat PERMANEN
func (c *Cart) UpdatePriceRight(newPrice float64) {
    c.TotalPrice = newPrice
}

func main() {
    myCart := Cart{TotalQty: 2, TotalPrice: 150000}

    myCart.UpdatePriceWrong(200000)
    fmt.Println("Salah:", myCart.TotalPrice) // Output tetap: 150000

    myCart.UpdatePriceRight(200000)
    fmt.Println("Benar:", myCart.TotalPrice) // Output berubah: 200000
}
```

2. Konstruktor di Go (Constructor Function)

Karena Go tidak memiliki method konstruktor bawaan seperti `constructor` di JS atau `__init__` di Python, konvensi standar di Go adalah membuat fungsi pembantu bernama `New[NamaStruct]` yang memvalidasi parameter input dan mengembalikan alamat memori (`*Struct`) dari objek baru.

```

package main

import (
    "errors"
    "fmt"
)

type Wallet struct {
    Owner    string
    Balance  float64
}

// Constructor Function: Mengembalikan pointer (*Wallet) agar hemat memori
func NewWallet(owner string, initialBalance float64) (*Wallet, error) {
    if owner == "" {
        return nil, errors.New("pemilik dompet tidak boleh kosong")
    }
    if initialBalance < 0 {
        return nil, errors.New("saldo awal tidak boleh negatif")
    }

    return &Wallet{
        Owner:    owner,
        Balance:  initialBalance,
    }, nil
}

func main() {
    myWallet, err := NewWallet("Budi", 500000)
    if err != nil {
        fmt.Println("Gagal:", err)
        return
    }
    fmt.Printf("Dompet milik %s berhasil dibuat dengan saldo: Rp %.2f\n", myWallet.Owner, myWallet.Balance)
}

```

Sesi 4: Praktikum Mandiri: "Sistem Logistik Pergudangan E-Commerce" (45 Menit)

Mari kita gabungkan Embedded Struct, Pointer, Constructor, dan Method untuk membangun simulator sistem logistik gudang e-commerce.

Implementasi Kode (`main.go`)

```
package main

import (
    "errors"
    "fmt"
)

// 1. Struct Address
type Address struct {
    Street  string
    City    string
    ZipCode string
}

// 2. Struct Vendor
type Vendor struct {
    ID      string
    Name    string
    Address Address // Komposisi Address
}

// 3. Struct Product
type Product struct {
    SKU      string
    Name     string
    Price    float64
    Stock    int
    Vendor   *Vendor // Menyimpan pointer ke Vendor (menghindari duplikasi objek vendor)
}

// 4. Constructor Function untuk Product
func NewProduct(sku, name string, price float64, stock int, vendor *Vendor) (*Product, error) {
    if sku == "" || name == "" {
        return nil, errors.New("SKU dan nama produk wajib diisi")
    }
    if price <= 0 {
        return nil, errors.New("harga produk harus lebih dari nol")
    }
    if stock < 0 {
        return nil, errors.New("stok awal tidak boleh negatif")
    }
    if vendor == nil {
        return nil, errors.New("produk harus memiliki vendor yang valid")
    }

    return &Product{
        SKU:      sku,
        Name:     name,
        Price:    price,
        Stock:    stock,
        Vendor:   vendor,
    }, nil
}
```



```

        Vendor: vendor,
    }, nil
}

// 5. Method dengan Pointer Receiver untuk menambah stok
func (p *Product) Restock(qty int) error {
    if qty ≤ 0 {
        return errors.New("kuantitas restock harus lebih dari nol")
    }
    p.Stock += qty // Mengubah nilai asli stock
    return nil
}

// 6. Method dengan Pointer Receiver untuk memotong stok saat penjualan
func (p *Product) DeductStock(qty int) error {
    if qty ≤ 0 {
        return errors.New("kuantitas penjualan harus lebih dari nol")
    }
    if p.Stock < qty {
        return fmt.Errorf("stok tidak mencukupi untuk SKU %s (Stok: %d, Dibutuhkan: %d)", p.SKU, p.Stock, qty)
    }
    p.Stock -= qty // Mengubah nilai asli stock
    return nil
}

// 7. Method dengan Value Receiver untuk menampilkan rincian produk (Read-only)
func (p Product) PrintDetails() {
    fmt.Println("-----")
    fmt.Printf("SKU           : %s\n", p.SKU)
    fmt.Printf("Nama Produk    : %s\n", p.Name)
    fmt.Printf("Harga Satuan   : Rp %.2f\n", p.Price)
    fmt.Printf("Stok Saat Ini  : %d unit\n", p.Stock)
    fmt.Printf("Dikirim Oleh   : %s (%s, %s)\n", p.Vendor.Name, p.Vendor.Address.Street, p.Vendor.Address.City)
    fmt.Println("-----")
}

func main() {
    // Inisialisasi Vendor & Alamat
    gudangUtama := &Vendor{
        ID: "VND-XYZ",
        Name: "PT Logistik Bersama",
        Address: Address{
            Street: "Kawasan Industri Blok B",
            City: "Bekasi",
            ZipCode: "17111",
        },
    }

    fmt.Println("=====")
    fmt.Println("                SISTEM GUDANG LOGISTIK E-COMMERCE                ")
    fmt.Println("=====")

    // Membuat produk baru melalui Constructor

```

```

    prod, err := NewProduct("SKU-ROG-01", "Asus ROG Phone", 15000000, 10, gudangUtam
    if err != nil {
        fmt.Println("Error Pembuatan Produk:", err)
        return
    }

    // Tampilkan data awal
    fmt.Println("Data Produk Baru Terdaftar:")
    prod.PrintDetails()

    // Simulasi Restock Gudang
    fmt.Println("\n>> Melakukan restock masuk sebanyak 5 unit...")
    if err := prod.Restock(5); err != nil {
        fmt.Println("Gagal Restock:", err)
    }
    fmt.Printf("Stok Baru: %d unit\n", prod.Stock)

    // Simulasi Penjualan (Potong Stok)
    fmt.Println("\n>> Terjadi penjualan sebanyak 3 unit...")
    if err := prod.DeductStock(3); err != nil {
        fmt.Println("Gagal Potong Stok:", err)
    }

    // Tampilkan rincian akhir produk
    fmt.Println("\nStatus Akhir Inventaris Produk:")
    prod.PrintDetails()
}

```

Sesi 5: Soal Latihan Praktik E-Commerce (Mandiri)

Selesaikan dua latihan pemrograman e-commerce tingkat lanjut di bawah ini untuk memperkuat pemahaman Anda mengenai integrasi Struct, Method, Pointer, dan penanganan kesalahan.

Latihan 1: Sistem Upgrade Level Loyalty Member E-Commerce

Deskripsi Soal: Buatlah sebuah sistem manajemen keanggotaan loyalty pelanggan e-commerce yang secara otomatis meningkatkan tingkatan (*tier*) keanggotaan berdasarkan akumulasi pembelian mereka.

Kriteria Implementasi:

1. Buat struct `ShippingAddress` (Jalan, Kota, Kode Pos).
2. Buat struct `CustomerProfile` yang memiliki field:
 - `ID` (string)
 - `Name` (string)
 - `LoyaltyTier` (string - default "Silver")

- `TotalSpent` (float64)
 - `Address` (pointer ke `ShippingAddress`)
3. Implementasikan method berikut pada `CustomerProfile` :
- `RecordTransaction(amount float64) error` : Method dengan **Pointer Receiver** untuk mencatat transaksi baru. Tambahkan `amount` ke `TotalSpent`. Evaluasi dan perbarui secara otomatis `LoyaltyTier` pelanggan dengan aturan:
 - Jika `TotalSpent` mencapai atau melebihi Rp 10.000.000: Upgrade ke `"Gold"`.
 - Jika `TotalSpent` mencapai atau melebihi Rp 50.000.000: Upgrade ke `"Platinum"`.
 - `UpdateAddress(newAddr ShippingAddress)` : Method dengan **Pointer Receiver** untuk memperbarui alamat pengiriman.
 - `PrintProfile()` : Method dengan **Value Receiver** untuk menampilkan data profil, tingkat loyalty saat ini, total belanja, dan alamat lengkap dengan rapi.
4. Ujilah alur di atas di dalam fungsi `main()` dengan mendaftarkan satu pelanggan baru, mengubah alamat pengirimannya, mencatat transaksi berturut-turut yang memicu upgrade tier keanggotaan dari Silver -> Gold -> Platinum, lalu tampilkan hasil profil terkini.
-

Latihan 2: Sistem Checkout & Potong Saldo e-Wallet (Mutasi Pointer)

Deskripsi Soal: Buatlah simulasi checkout e-commerce yang menghubungkan struct transaksi dengan saldo e-Wallet pelanggan secara dinamis menggunakan pointer.

Kriteria Implementasi:

1. Buat struct `EWallet` (`OwnerName`, `Balance`).
2. Buat struct `OrderTransaction` (`OrderID`, `ItemTotal`, `Status`).
3. Implementasikan method `Pay(wallet *EWallet) error` pada `OrderTransaction` :
 - Method ini menerima pointer ke `EWallet`.
 - Lakukan validasi: Jika saldo dompet (`wallet.Balance`) kurang dari total tagihan transaksi (`ItemTotal`), ubah status transaksi menjadi `"FAILED"` dan kembalikan error `"pembayaran gagal: saldo e-wallet tidak mencukupi"`.
 - Jika saldo cukup, potong langsung saldo dompet (`wallet.Balance` dikurangi `ItemTotal`), lalu ubah status transaksi menjadi `"PAID"`.
4. Buat fungsi konstruktor `NewOrderTransaction(id string, total float64) (*OrderTransaction, error)` untuk memastikan total belanja tidak boleh kurang dari atau sama dengan nol.
5. Di dalam fungsi `main()`, buat satu dompet digital `EWallet` dengan saldo Rp 5.000.000. Lakukan dua simulasi transaksi menggunakan pointer dompet yang sama:
 - **Transaksi A:** Total belanja Rp 1.500.000. Tampilkan status transaksi dan sisa saldo e-wallet setelah transaksi selesai diproses.
 - **Transaksi B:** Total belanja Rp 4.000.000. Buktikan transaksi ini gagal diproses akibat sisa saldo tidak cukup, dan pastikan saldo dompet tidak berubah (tidak terpotong).